CS 344 **Problem Set 2**
Fall 2021 **Due:** October 25, 2021, 11:59 p.m.

Name: (Xiaowei Zhang) **NetID**: (xz561)

Honor Code: Students may discuss and work on homework problems in groups, which is encouraged. However, each student must write down their solutions independently to show they understand the solution well enough to reconstruct it by themselves. Students should clearly mention the names of the other students who offered discussions. We check all submissions for plagiarism. We take the honor code seriously and expect students to do the same.

Instruction for Submission: This homework has a total of 100 points, it will be rescaled to 10 points as the eventual score.

We encourage you to use LaTex to write your answer, because it's particularly suitable to type equations and it's frequently used for writing academic papers. We have provided the homework1.tex file for you, you can write your answer to each question in this .tex file directly after the corresponding question, and then compile the .tex file into a PDF file for submission. As a quick introduction to LaTex, please see this *Learn LaTeX in 30 minutes* [1]. Compiling a .tex file into a PDF file needs installing the LaTex software on your laptop. It's free and open source. But if you don't want to install the software, you can just use this website `https://www.overleaf.com/`, which is also free of charge. You can just create an empty project in this website and upload the homework1.zip, and then the website will compile the PDF for you. You can also directly edit your answers on the website and instantly compile your file.

You can also use Microsoft Word or other software if you don't want to use LaTex. If so, please clearly number your answers so that we know which answer corresponds to which question. You also need to save your answers as a PDF file for submission.

Finally, please submit your PDF file only. You should name your PDF file as "Firstname-Lastname-NetID.pdf''.

Late Policy: The homework is due on 10/25 (Monday) at 11:59pm. We will release the solutions of the homework on Canvas on 10/29 (Friday) 11:59pm. If your homework is submitted to Canvas before 10/25 11:59pm, there will no late penalty. If you submit to Canvas after 10/25 11:59pm and before 10/29 11:59pm (i.e., before we release the solution), your score will be penalized by 0.9^k, where k is the number of days of late submission. For example, if you submitted on 10/28, and your original score is 80, then your final score will be $80 * 0.9^3 = 58.32$ for $28 - 25 = 3$ days of late submission. If you submit to Canvas after 10/29 11:59pm (i.e., after we release the solution), then you will earn no score for the homework.

Make best use of picture in Latex: If you think some part of your answer is too difficult to type using Latex, you may write that part on paper and scan it as a picture, and then insert that part into Latex as a picture, and finally Latex will compile the picture into the final PDF

[1] `https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes`

output. This will make things easier. For instructions of how to insert pictures in Latex, you may refer to the Latex file of our homework 1, which includes several examples of inserting pictures in Latex.

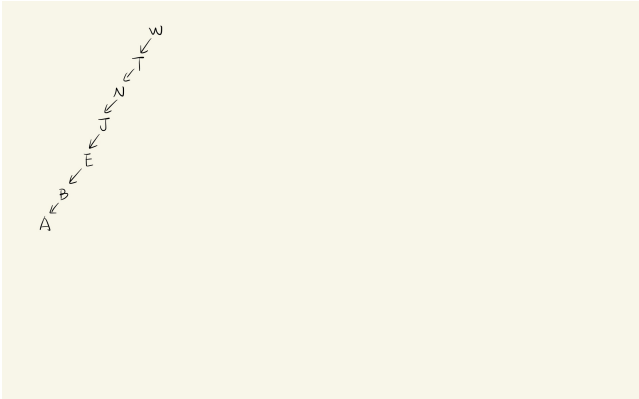
Discussion Group (People with whom you discussed ideas used in your answers if any):

I acknowledge and accept the Honor Code. Please type your initials below:
Signed: (XZ)

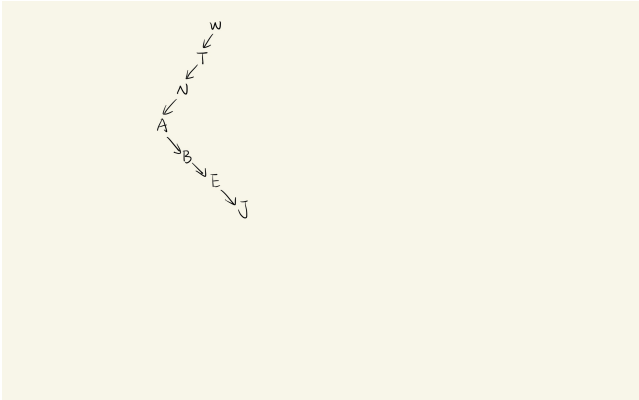
---

1. **Warm up with BST.** (20 points) Beginning with an empty binary search tree, what binary search tree is formed when you insert the following values in the order given? (Values are compared in alphabet order, e.g., $A < B$)

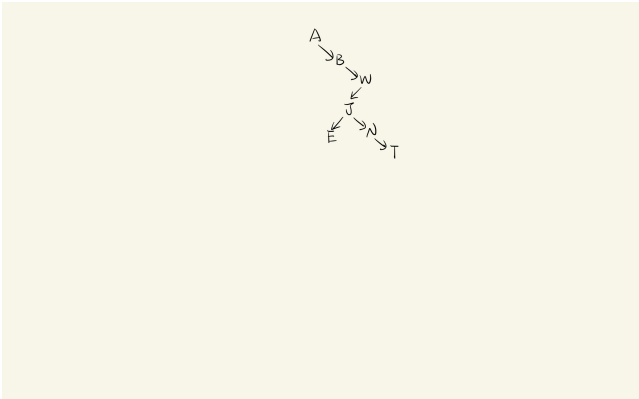
   (a) W, T, N, J, E, B, A

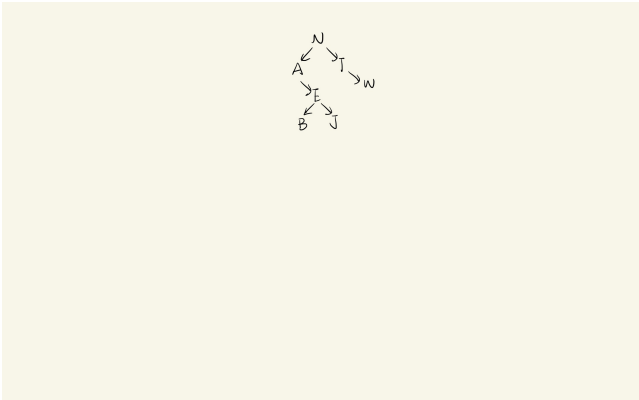
   (b) W, T, N, A, B, E, J

   (c) A, B, W, J, N, T, E



(d) N, A, T, E, B, W, J



2. **Binary search tree with equal keys.** (40 points) In the class, we assumed that all the keys in a BST are different from each other. However, the BST can still store keys of the same value, in this case, we put a key that is less than a node to its left, and put a key that is greater than **or equal to** a node to its right. Here is the algorithm for inserting a new key z in to a binary search tree T:

Algorithm 1: Tree-Insert(T, z)

```
y = NIL;
x = T.root;
while x ≠ NIL
    y=x;
    if z.key<x.key
        x = x.left;
    else x = x.right
z.parent = y;
if y==NIL
    T.root = z;    //tree T was empty
elseif z.key<y.key
    y.left=z;
else y.right=z;
```

(a) (10 points) To better understand the algorithm, draw the tree generated by inserting the numbers 5, 3, 10, 8, 12, 12, 8, 8, 6, 6, 7, 5, 8 in this given order into an initially empty binary search tree using the above algorithm.

(b) (10 points) What is the asymptotic runtime of TREE-INSERT when used to insert n items with identical keys into an initially empty binary search tree?

Answer: The asymptotic runtime is $O(n^2)$, because the resulting binary search tree is extremely unbalanced and the height of the tree will be n. It will take $O(n)$ to traverse to the bottom of the tree to insert one item. There are n items. Therefore, the total runtime would be $O(n^2)$ in this case

We propose to improve TREE-INSERT by testing before line 5 to determine whether z.key==x.key and by testing before line 11 to determine whether z.key==y.key. If equality holds, we implement one of the following strategies. For each strategy, find the asymptotic runtime of inserting n items with identical keys into an initially empty binary search tree. (The strategies are described for line 5, in which we compare the keys of z and x, and substitute y for x to arrive at the strategies for line 11.)

(c) (10 points) Keep a list of nodes with equal keys at x, and insert z into the list. Answer: This will resulting a list of n elements while the height of the tree remains one, i.e. only root. Because the program iterates the while loop only once, a single insertion will be done in constant time, and n items will be done in linear time. That is, $O(n)$

(d) (10 bonus) Randomly set x to either x.left or x.right. (Give the worst-case runtime and informally derive the expected runtime.)

Answer:

Worst Case: every random choice sets x to left, and it will take $O(n^2)$, the same as original

Expected runtime:because x is randomly chosen to be set to x.left or x.right, the resulting tree should be pretty much balanced. For a balanced tree, a single insertion will require $O(logn)$ as the depth will be approximately $logn$, and n items will require $O(nlogn)$

[**Note: for (b) (c) and (d), we are expecting the runtime represented as a function of** n].

3. **Hat-check problem.** (10 points) Use indicator random variables to solve the following

problem, which is known as the hat-check problem. Each of n customers gives a hat to a hat-check person at a restaurant. The hat-check person gives the hats back to the customers in a random order. What is the expected number of customers who get back their own hat?

Answer: Let X_i be the indicator random variable of the event the i-th customer gets his own hat back. That is,

$X_i = 1$ if the ith person gets his own hat, 0 otherwise

The probability for one particular customer to gets his own hat back is $P(X_i = 1) = \frac{1}{n}$. Let X denotes the total number of customers who get back their own hats. Therefore, $E(X) = E(\sum_{i=1}^{n} X_i)$. Because of the linearity of expectation, the expectation of sum of n random variables equals to the sum of expectations of random variables. Therefore, $E(X) = \sum_{i=1}^{n} E(X_i)$. $E(X_i) = P(X_i = 1) * X_i + P(X_i = 0) * X_i = \frac{1}{n}$. Thus $E(X) = \sum_{i=1}^{n} \frac{1}{n} = 1$. That is, the expected number of customers who get back their own hat is 1

4. **Quicksort with equal element values.** (30 points) The analysis of the expected running time of randomized Quicksort in class (corresponding to Section 7.4.2 in textbook) assumes that all element values are distinct. In this problem, we examine what happens when they are not, i.e., there exist same-valued elements in the array to be sorted.

   The Quicksort algorithm relies on the following partition algorithm, which finds a pivot randomly, and then put all the numbers less than or equal to the pivot in the left, and put all the numbers greater than the pivot in the right, and then return the pivot location, as well as the left and right sublists for recursive calls.

Algorithm 2: Partition(A, p, r)

```
q=RANDOM(p,r);  //generate a random number in the range of [p,r]
L=empty list, R=empty list;
for each element a in A except A[q]:
    if a≤A[q]:
        append a to L;
    else append a to R;
A=append(L,A[q],R);
returen A, q;
```

In this algorithm, the list to be sorted is A, and we use p and r to denote the left-most and right-most indices of the currently processing subarray, respectively. For example, in the initial call of the Quicksort algorithm, we will let $p = 0$ and $r = n-1$, which correspond to the whole original array. The PARTITION(A, p, r) procedure returns an index q such that each element of $A[p : q-1]$ is less than or equal to $A[q]$ and each element of $A[q+1 : r]$ is greater than $A[q]$.

(a) (10 points) Suppose there are n elements and all element values are equal. What would be randomized Quicksort's running time in this case?

Answer: Since all n elements are equal to each other, that means for every element, it will be compared to every other elements exactly once because R will always be empty for every partition. So the running time of the randomized quick-sort in this case should be $O(n^2)$

(b) (10 points) Modify the PARTITION procedure to produce a procedure PARTITION$'(A, p, r)$, which permutes the elements of $A[p : r]$ and returns two indices q and t, where $p \leq q \leq t \leq r$, such that:

i. all elements of $A[q : t]$ are equal,

ii. each element of $A[p : q - 1]$ is less than $A[q]$, and

iii. each element of $A[t + 1 : r]$ is greater than $A[q]$

Like PARTITION, your PARTITON$'$ procedure should take $O(r - p)$ time.

Answer:

```
Patition'(A,p,r):
q=RANDOM(p,r)
L=empty list, R=empty list, M=empty list;
for each elements a in A except A[q]:
if a<A[q]: append a to L;
elif a==A[q]: append a to M;
else: append a to R;
t = q+length(M)
A=append(L,A[q],M,,R)
return q,t
```

(c) (10 points) Now you have a PARTITION$'(A, p, r)$ algorithm, based on this algorithm, provide the pseudocode of a QUICKSORT$'(A)$ algorithm which calls PARTITION$'(A, p, r)$ as a subroutine, so that it recurses only on partitions of elements not known to be equal to each other. [Hint: it will looks very similar to our Quicksort pseudocode in the lecture slides, you only need to make minor modifications to recurse on the partitions produced by PARTITION$'(A, p, r)$.]

Answer:

```
QuickSort'(A):
if length(A) ≤ 1: return
p = 0, r = length(A)-1
q,t = Partition'(A,p,r)
if p≤q-1
L = A[p:q-1]
QuickSort'(L)
if t+1≤r
R = A[t+1:r]
QuickSort'(R)
```